C05 AT 3

Flowchart to code conversion

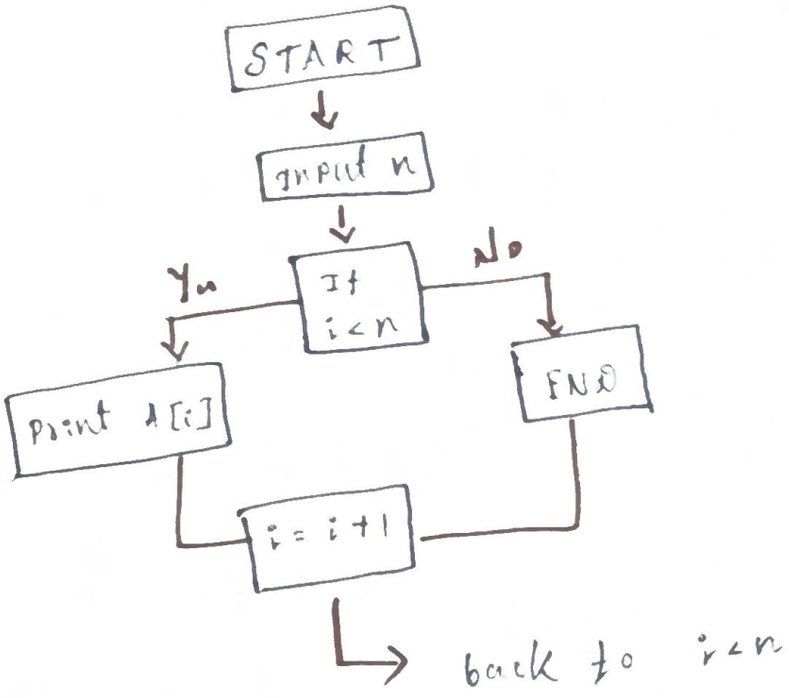
Name: Manoj M

Reg no: 192511061

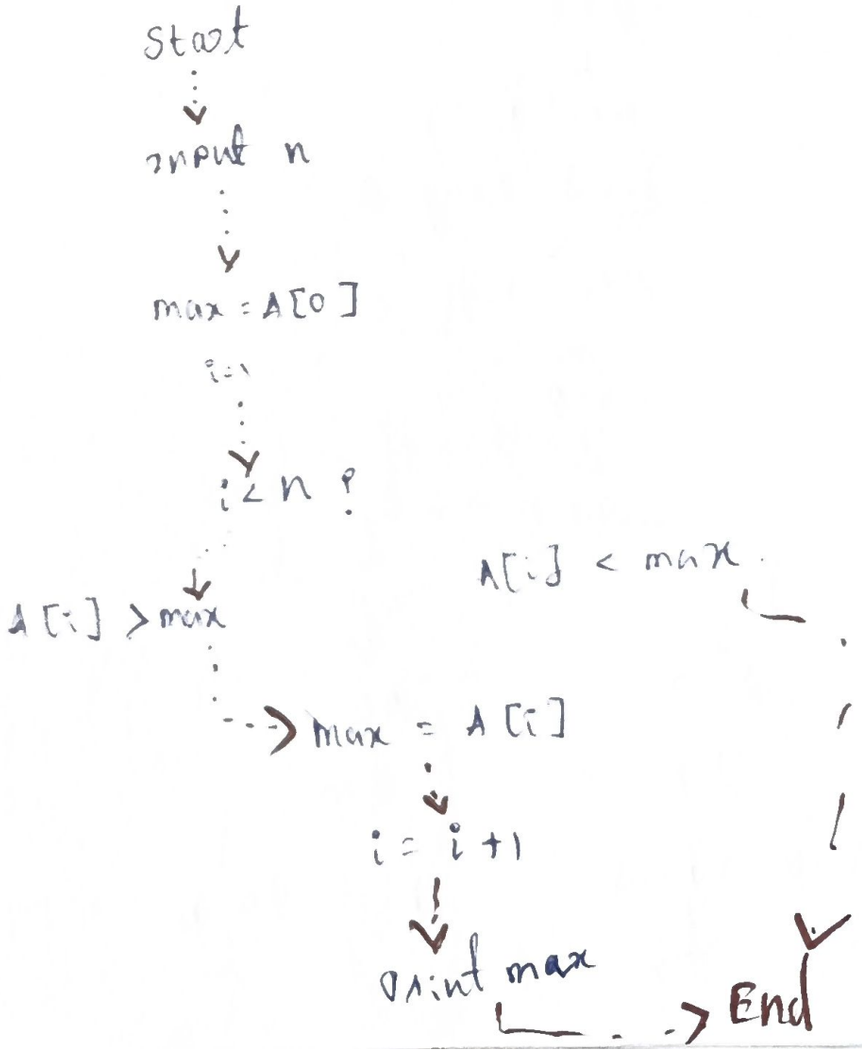
Course Name: Data structures

Course code: CSA0301

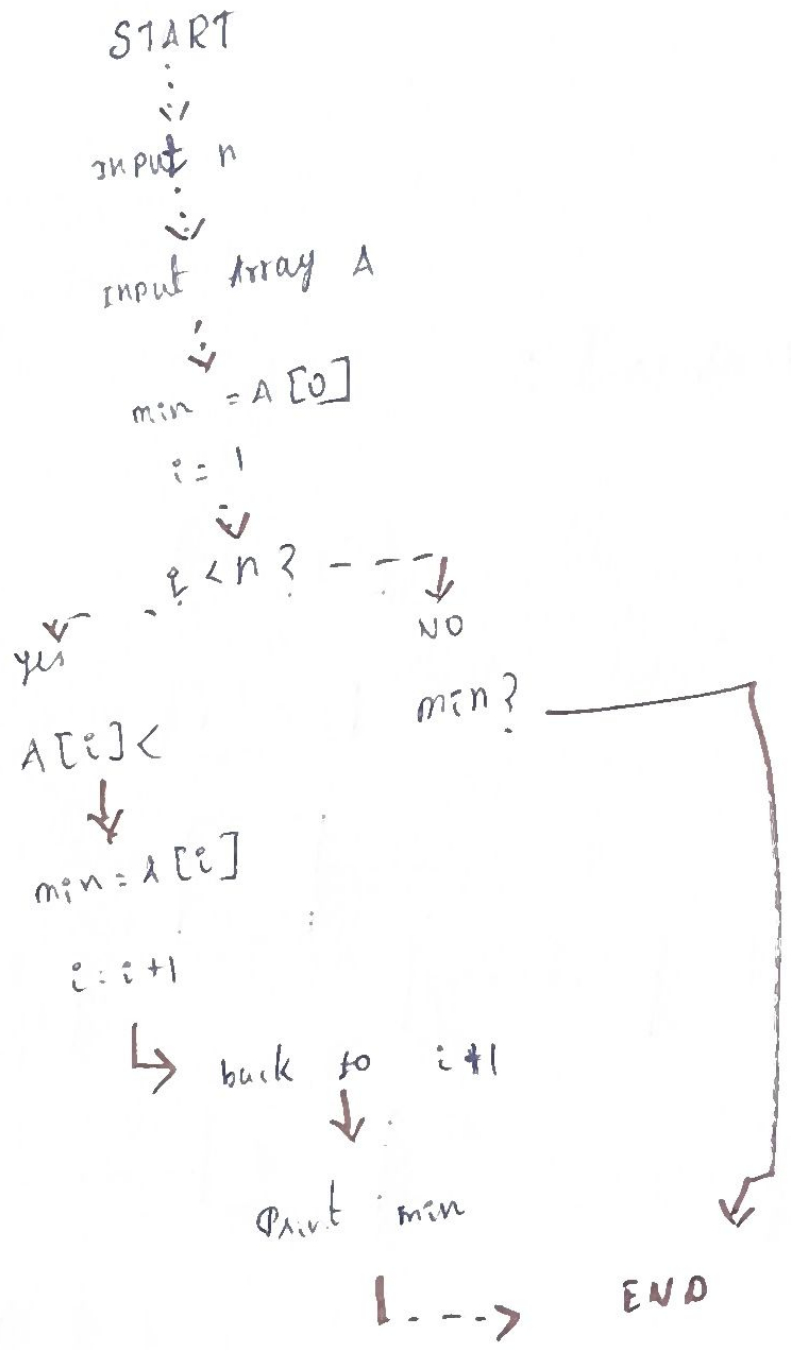
1. Array Traversal :-



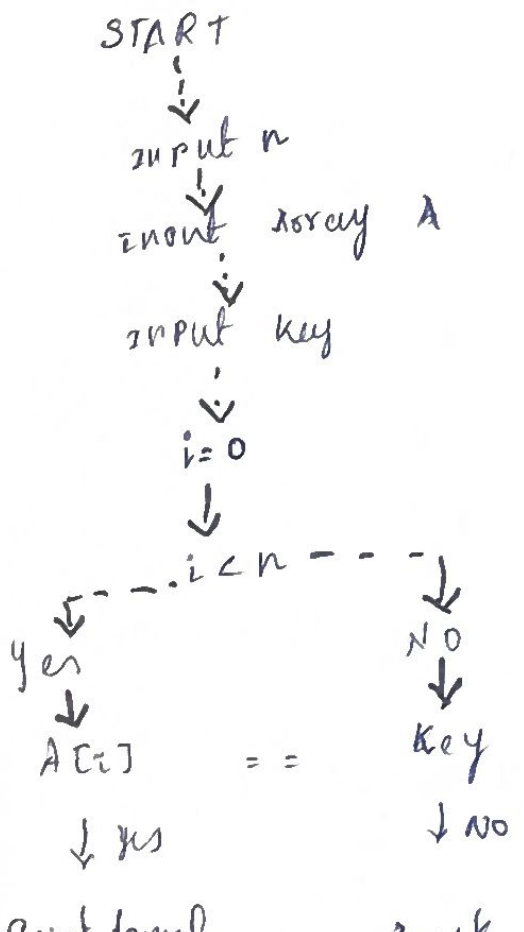
2. Find Largest Element



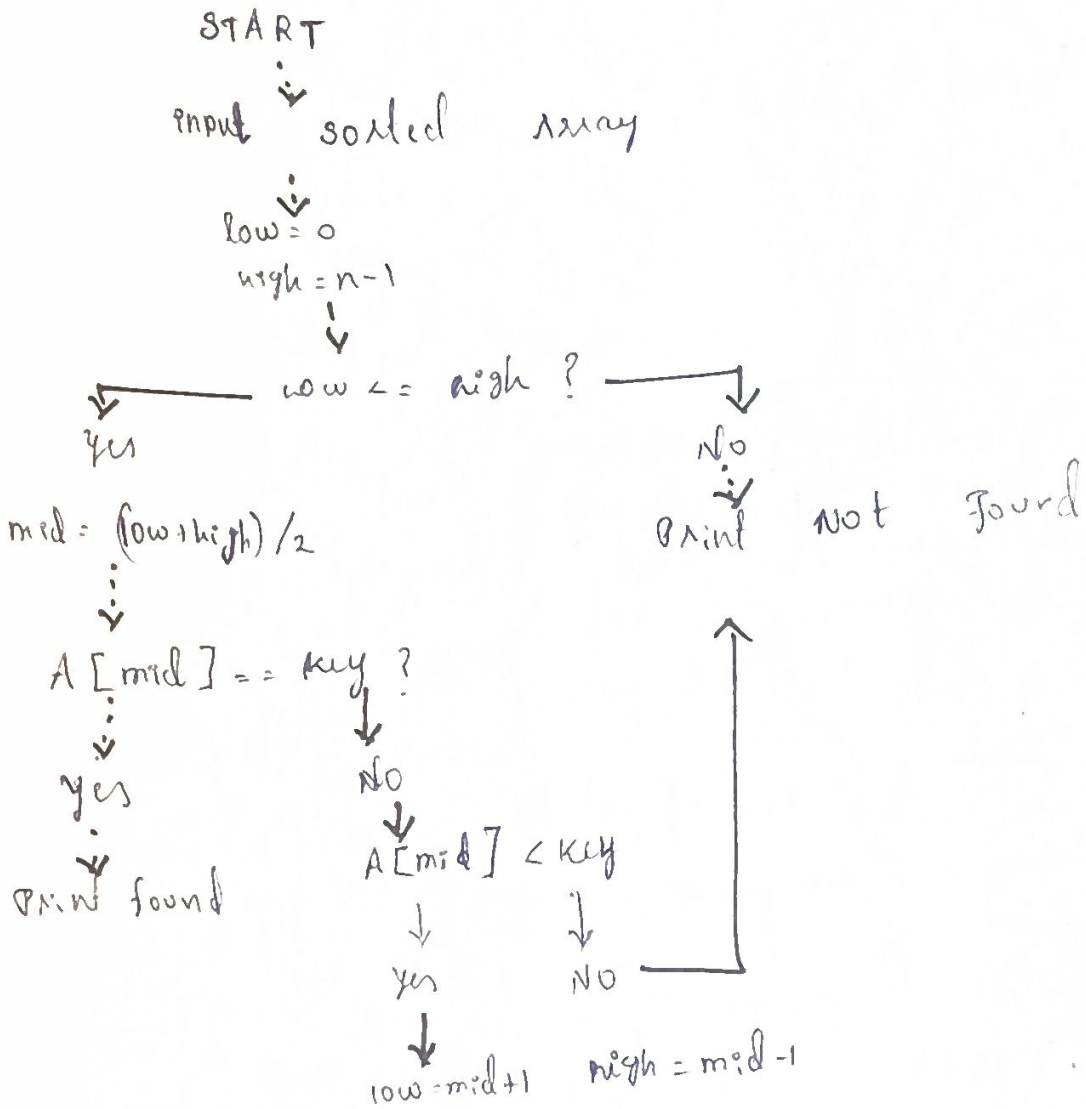
3. Find smallest



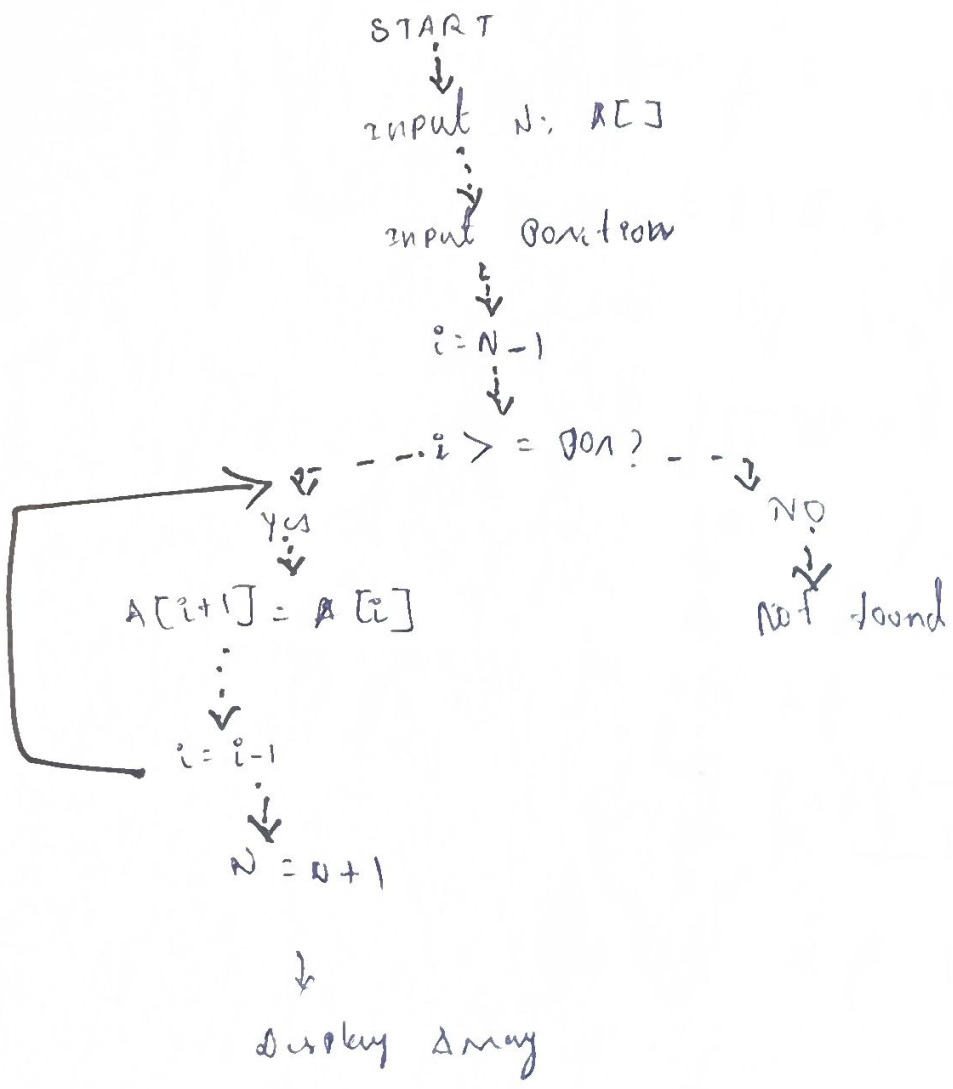
4. Linear search



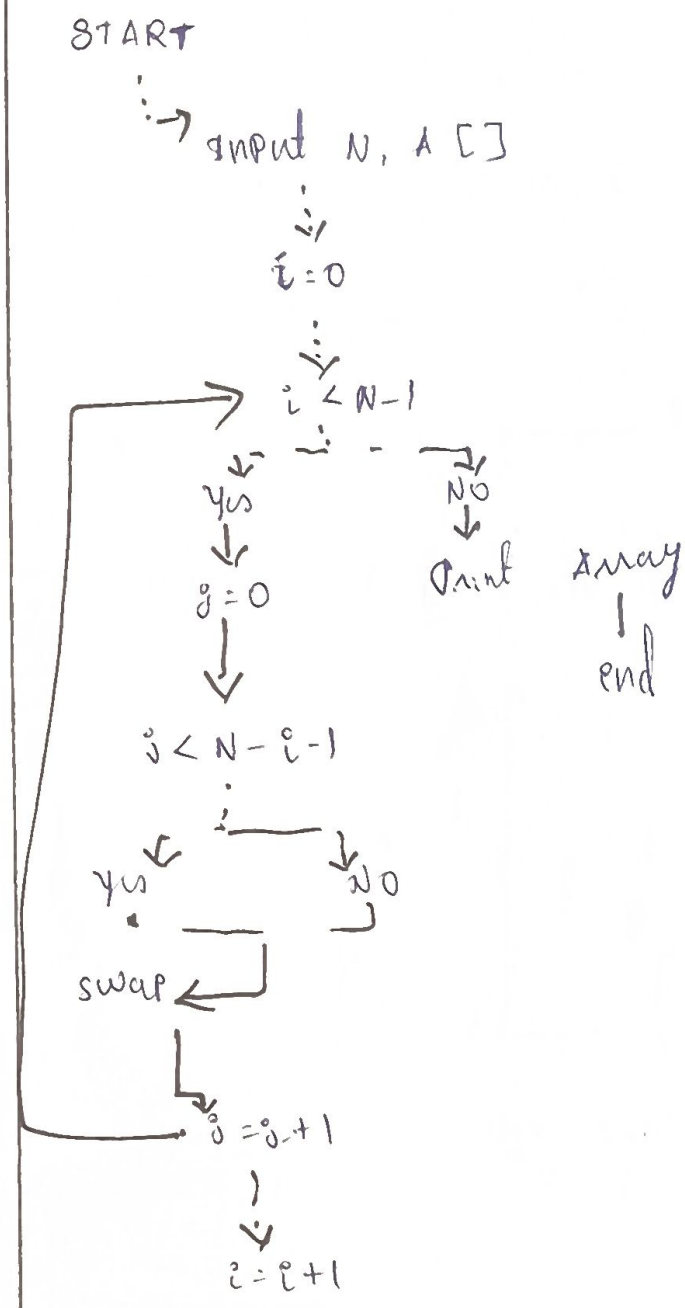
5. Binary search



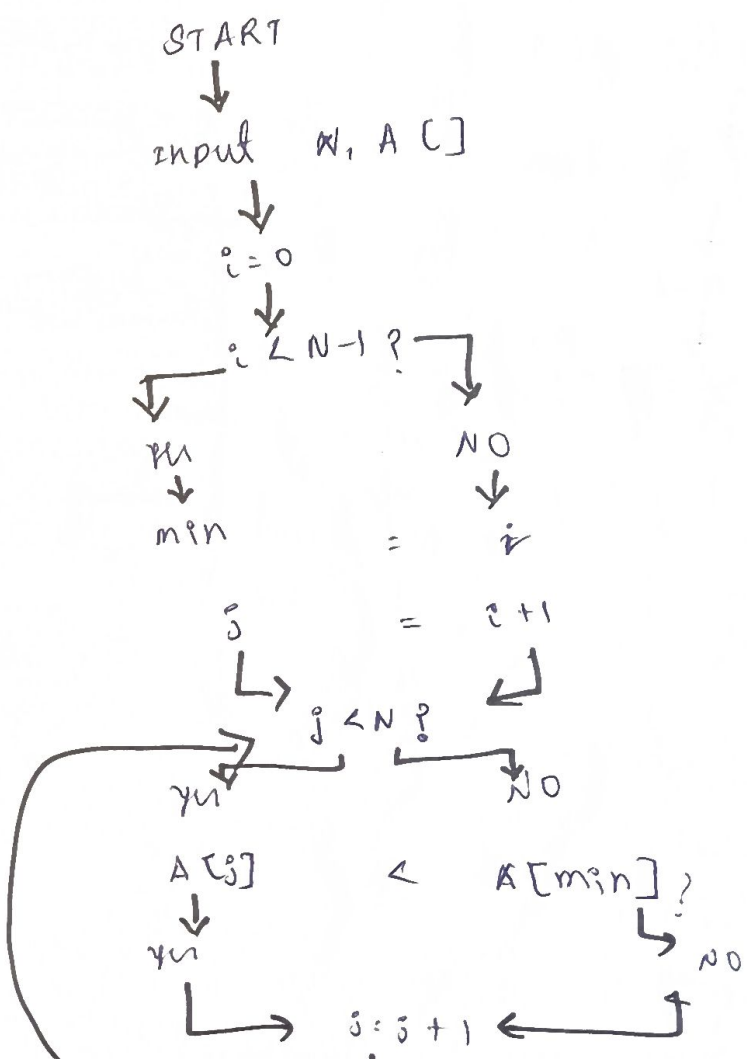
6. Insert element in Array



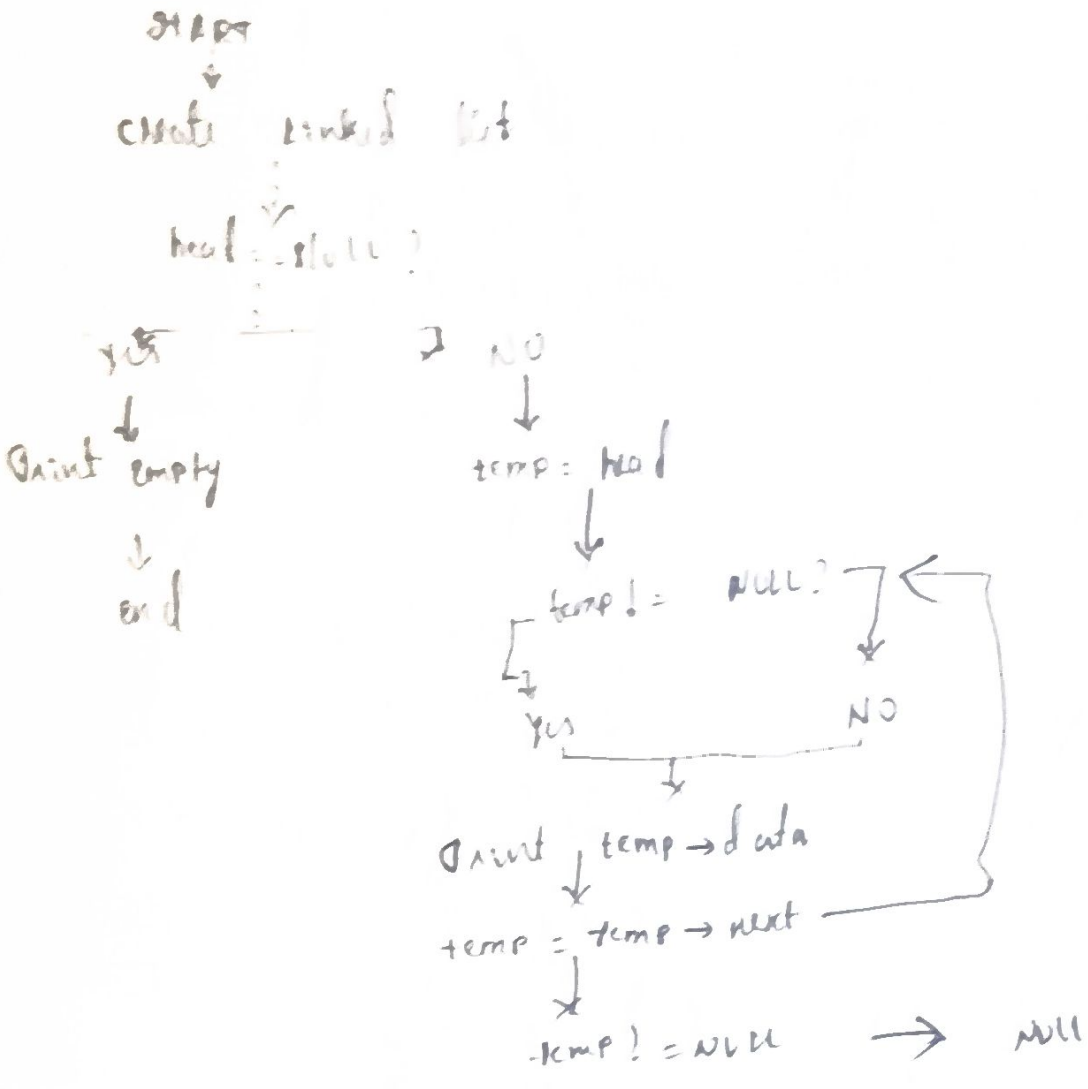
7. Bubble sort



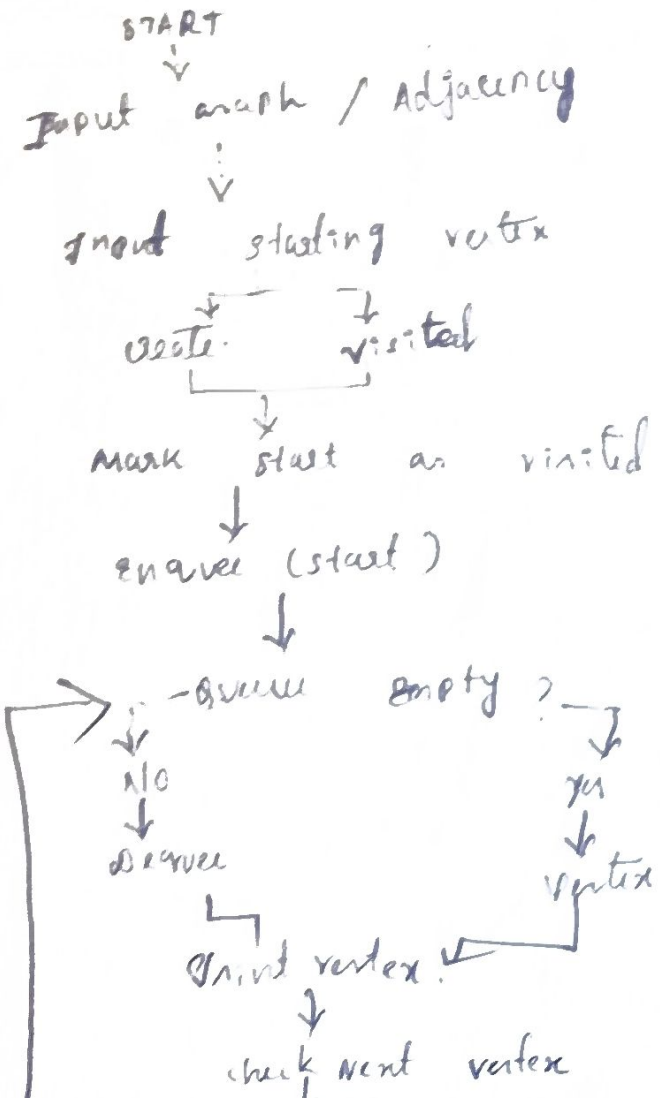
8. selection sort



...ing linked list traversal.



BFS Using Queue



1. Array traversal & Displaying elements

```
# include <stdio.h>
int main() {
    int arr[] = {10, 20, 30, 40}
    int n = 5;
    printf("Array elements");
    for (int i=0; i<n; i++) {
            printf("%d ", arr[i]);
    }
}
```

2. Find the Largest Element in an array

```
# include <stdio.h>
int main() {
    int arr[] = {12, 16, 13, 12, 11};
    int n = 5;
    int max = arr[0];
    for (int i=0; i<n; i++) {
        if (arr[i] > arr[max]) {
            max = arr[i];
        }
    }
    return 0;
}
```

```
#include <stdio.h>
int main() {
    int arr[] = {64, 34, 25, 12};
    int n=5, temp;
    for(int i=0; i<n-i; i++){
        for(int j=0; j<n-i-1; j++){
            if(arr[j] > arr[j+1]){
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
}   }   {
```

6. Binary search

```
#include <stdio.h>
int main() {
    int arr[] = {10,20,30,40,50};
    int n= 5 , key =40 ;
    int low=0 ; high = n-1 ,mid , found=0;
    while ( low <= High) {
        mid = low + (high-low) /2 ;
        IF (arr[mid] == key) {
            found = 1;
        }       break;
        else if (arr[mid] < key) {
            low = mid
```

```
    }
  }
  if (!found) {
      printf ("not found.");
    }
  }
```

7. Singly Linked List Traversal

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

int main() {
    struct Node *head = (struct Node*) malloc (sizeof(struct Node));
    struct Node *second = (struct Node*) malloc(sizeof(struct Node));

    head -> data = 100;
    head -> next = second;

    second -> data = 200;
    second -> next = NULL;

    struct Node* temp = head;
    while (temp != NULL) {
        printf ("%d -> ", temp -> data);
        temp = temp -> next;
    }

    free (head);
    free (head);

    return 0;
}
```

stack overflow and underflow checks.

```
#include <stdio.h>
#define MAX 2
int stack[MAX];
int top=-1;
void push(int val){
    if (top==Max-1){
        printf("stack overflow");
    }
    else{
        stack[++top]=val;
        printf("pushed", val);
}   {
if (pop==-1){
    printf("underflow");
    }
int main(){
    pop();
    push(10);
    push(20);
```

Queue operations (enqueue & dequeue)

```
#include <stdio.h>
#define Max 5
int queue[MAX];
int front=-1; rear=-1;
void enqueue(int val){
    if (rear==Max-1){
        printf("Que overflow");
    }
```

```
}
void dequeue(){
    if (front == -1 || front > rear) {
        printf("Queue underflow");
    }
    else{
        printf("Dequeue %d", queue[front++]);
    }
}
int main(){
    enqueue(5);
    enqueue(15);
}
```

10. Delete First Node from a singly Linked list

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void delete(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}
int main() {
    struct Node* head = (struct Node*) malloc(sizeof(struct Node));
    head->data = 10;
    head->next = NULL;
    free(head);
}
```